

Izaz Ahmed Asif
Student ID:- 16372207

Distributed Master–Worker System using FastAPI (RESTful Implementation)

1. Introduction

In this project, I implemented a distributed master–worker architecture using the RESTful approach with FastAPI. The goal was to design a system where a client can query information about individuals, and the request is processed through a master server that forwards it to worker servers.

2. System Architecture

The system consists of four components:

client_rest.py (REST-based client)

master_api.py (FastAPI server on port 23000)

worker_api.py/1 (handles names a–m, port 23001)

worker_api.py/2 (handles names n–z, port 23002)

The architecture follows this flow:

Client → Master → Worker(s) → Master → Client

3. Client Implementation

1. Accepts the master port as a command-line argument.
2. Sends HTTP GET requests using the httpx library.
3. Calls:
 - 3.1 /getbyname
 - 3.2 /getbylocation
 - 3.3 /getbyyear
4. Prints the JSON response.
5. The client includes exception handling for:
 - 5.1 Read timeouts
 - 5.2 General connection errors

4. Worker Implementation

Endpoints:

/getbyname - checks if the name exists in its local dictionary.

/getbylocation - iterates over records and returns matching locations.

/getbyyear - filters by both location and year.

/health - checks if master and worker are working or not

5. Master Implementation

5.1 Request Routing

For /getbyname:

5.1.1 If the name starts with a–m → forwarded to Worker 1.

5.1.2 If the name starts with n–z → forwarded to Worker 2.

For /getbylocation and /getbyyear:

5.1.3 The master sends requests to both workers.

5.1.4 It combines the results before returning them to the client.

5.2 Fault Tolerance

The master uses a helper function called `safe_get()` to call workers. If anything happens like Is down, Times out, or throws an error it doesn't crush. It returns an empty result for that worker and continues operating.

6. Observations

1. When all servers are running, the system returns complete results.

2. If Worker 1 is stopped:

2.1 Queries for names a–m fails.

2.2 Location and year queries return partial results from Worker 2.

3. If Worker 2 is stopped:

3.1 Queries for names n–z fail.

3.2 Location and year queries return partial results from Worker 1.

4. If both workers are stopped:

4.1 The master still runs.

4.2 All queries return empty results.

4.3 The system does not crash.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS D:\Distributed> $env:WORKER_GROUP="am"
PS D:\Distributed> uvicorn worker_api:app --host 127.0.0.1 --port 23001
INFO: Started server process [15624]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://127.0.0.1:23001 (Press CTRL+C to quit)
INFO: 127.0.0.1:62910 - "GET /getbylocation?location=Kansas+City HTTP/1.1" 200 OK
INFO: 127.0.0.1:62913 - "GET /getbyyear?location=New+York+City&year=2002 HTTP/1.1" 200 OK
```

Worker (am) server running on port 23001

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS D:\Distributed> $env:WORKER_GROUP="nz"
PS D:\Distributed> uvicorn worker_api:app --host 127.0.0.1 --port 23002
INFO: Started server process [15380]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://127.0.0.1:23002 (Press CTRL+C to quit)
INFO: 127.0.0.1:62908 - "GET /getbyname?name=xander HTTP/1.1" 200 OK
INFO: 127.0.0.1:62911 - "GET /getbylocation?location=Kansas+City HTTP/1.1" 200 OK
INFO: 127.0.0.1:62914 - "GET /getbyyear?location=New+York+City&year=2002 HTTP/1.1" 200 OK
```

Worker (nz) server running on port 23002

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS D:\Distributed> uvicorn master_api:app --host 127.0.0.1 --port 23000
INFO:      Started server process [15516]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
INFO:      Uvicorn running on http://127.0.0.1:23000 (Press CTRL+C to quit)
INFO:      127.0.0.1:62907 - "GET /getbyname?name=xander HTTP/1.1" 200 OK
INFO:      127.0.0.1:62909 - "GET /getbylocation?location=Kansas+City HTTP/1.1" 200 OK
INFO:      127.0.0.1:62912 - "GET /getbyyear?location=New+York+City&year=2002 HTTP/1.1" 200
OK
```

Master API successfully running on port 23000

```
Windows PowerShell
PS D:\Distributed> python client_rest.py 23000

Client => Person xander
{'error': False, 'result': [{'record_id': 9, 'name': 'xander', 'location': 'Los Angeles', 'year': 2005}], 'partial': False}

Client => Location Kansas City
{'error': False, 'result': [{'record_id': 4, 'name': 'bill', 'location': 'Kansas City', 'year': 2002}, {'record_id': 12, 'name': 'deep', 'location': 'Kansas City', 'year': 2022}, {'record_id': 1, 'name': 'rakin', 'location': 'Kansas City', 'year': 2019}, {'record_id': 3, 'name': 'zen', 'location': 'Kansas City', 'year': 2018}, {'record_id': 4, 'name': 'symon', 'location': 'Kansas City', 'year': 2015}, {'record_id': 14, 'name': 'niko', 'location': 'Kansas City', 'year': 2001}], 'partial': False}

Client => NYC in 2002
{'error': False, 'result': [{'record_id': 2, 'name': 'alice', 'location': 'New York City', 'year': 2002}], 'partial': False}
PS D:\Distributed>
```

Client REST application successfully sending requests to the Master API